```
  it 'returns a blank array when there are no matching expenses' do
    expect(ledger.expenses_on('2017-06-10')).to eq([])
  end
end
```

The overall flow—adding three expenses and then searching for two of them by date—looks similar to what we did in the acceptance spec in *Querying the Data*, on page 54. Here, you're testing at the level of the individual `Ledger` object, rather than the entire app.

We're using another composable matcher here to combine two matchers you've used before—`contain_exactly` and `a_hash_including`—into a new matcher that describes a nested data structure.

Run the spec file to make sure both specs fail:

```
$ bundle exec rspec spec/integration/app/ledger_spec.rb

Randomized with seed 3824

ExpenseTracker::Ledger
  #record
    with a valid expense
      successfully saves the expense in the DB
    when the expense lacks a payee
      rejects the expense as invalid
  #expenses_on
    returns a blank array when there are no matching expenses (FAILED - 1)
    returns all expenses for the provided date (FAILED - 2)

Failures:

  1) ExpenseTracker::Ledger#expenses_on returns a blank array when there ↩
  are no matching expenses
     Failure/Error: expect(ledger.expenses_on('2017-06-10')).to eq([])

       expected: []
            got: nil

       (compared using ==)
     # ./spec/integration/app/ledger_spec.rb:59:in `block (3 levels) in ↩
     <module:ExpenseTracker>'
     # ./spec/support/db.rb:9:in `block (3 levels) in <top (required)>'
     # ./spec/support/db.rb:9:in `block (2 levels) in <top (required)>'

  2) ExpenseTracker::Ledger#expenses_on returns all expenses for the ↩
  provided date
     Failure/Error:
       expect(ledger.expenses_on('2017-06-10')).to contain_exactly(
         a_hash_including(id: result_1.expense_id),
         a_hash_including(id: result_2.expense_id)
       )
```